University of East London

Image Classification Using CNN on Leaf Images Dataset

https://www.kaggle.com/datasets/ichhadhari/leaf-images

University of East London

Table of Contents

1. Introduction 3

1.1 Background and Motivation 3

1.2 Objectives of the Project 3

1.3 Relevance to Real-World Applications 3

1.4 Report Structure Overview 4

2. Creative and Innovative Approaches 5

2.1 Dataset Processing 5

2.2 Preprocessing Strategy and Read Function Customization 5

2.3 Justification of Approach for the Dataset 6

2.4 Comparison with Alternative Strategies 7

3. Simulations 8

3.1 Dataset Characteristics and Organization 8

3.2 Data Splitting and Subdivision 8

3.3 CNN Architecture Details 8

3.4 Training Configuration and Environment 10

4. Results Obtained 11

4.1 Overall Classification Accuracy 11

4.2 Confusion Matrix Interpretation 11

4.3 ROC Curve and Model Confidence 12

4.4 Per-Class Accuracy and Visual Checks 13

5. Critical Analysis of Results 14

5.1 Factors Affecting Model Performance 14

5.2 Potential Improvements to the Approach 15

6.Conclusions 16

6.1 Summary of Achievements 16

6.2 Limitations and Challenges 16

6.3 Recommendations for Future Work 16

7. References 17

1. Introduction

1.1 Background and Motivation

Over the last few years, image classification has become an essential part of nearly every computer vision application. One such domain is the classification of plant leaves for perhaps agricultural assessment, botanical or biodiversity surveys, or even use in diagnosing plant disease. I believe that applying AI to this task also minimizes the amount of human involvement at an expert level and enhances productivity. The significance of developing good classification models is therefore anchored on their ability to foster sustainable agriculture and advance research into environmental management. Classification of leaves is more complex because of variations in size, texture, color, and image capturing conditions, and for this reasons qualifies to provide a good testbed to assess deep learning algorithms such as CNN.

1.2 Objectives of the Project

[redacted]

[redacted]

[redacted]

[redacted]

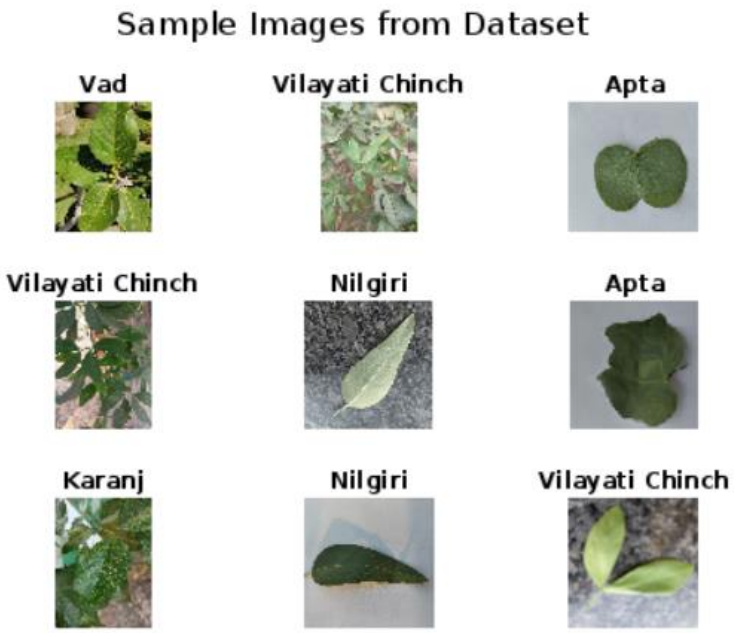
```
clc; clear; close all;

% Suppress non-critical warnings
warning('off','images:png:invalidChunk');

% Define dataset path
datasetPath = fullfile('300_dataset');

% Create an imageDatastore with specified file extensions
imds = imageDatastore(datasetPath, ...
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames', ...
    'FileExtensions', {'.jpg', '.jpeg', '.png'});
```

[redacted]

```
figure;
perm = randperm(length(imds.Files), 9);
for i = 1:9
    subplot(3,3,i);
    imshow(readimage(imds, perm(i)));
    title(string(imds.Labels(perm(i))));
end
sgtitle('Sample Images from Dataset');
saveas(gcf, 'figure1_sample_images.png');
% Define input size compatible with AlexNet-style CNN
inputSize = [227 227 3];
```

[redacted]

[redacted]



[redacted]

[redacted]

```
layers = [
    imageInputLayer(inputSize)

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,16,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,32,'Padding','same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(numel(unique(imds.Labels)))
    softmaxLayer
    classificationLayer];
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
% Assign custom read function with error handling
imds.ReadFcn = @(filename) preprocessImage(filename, inputSize(1:2));

% Split dataset: 64% Train, 16% Validation, 20% Test
[imdsTrain, imdsTest] = splitEachLabel(imds, 0.8, 'randomized');
[imdsTrain, imdsVal] = splitEachLabel(imdsTrain, 0.8, 'randomized');
```

[redacted]

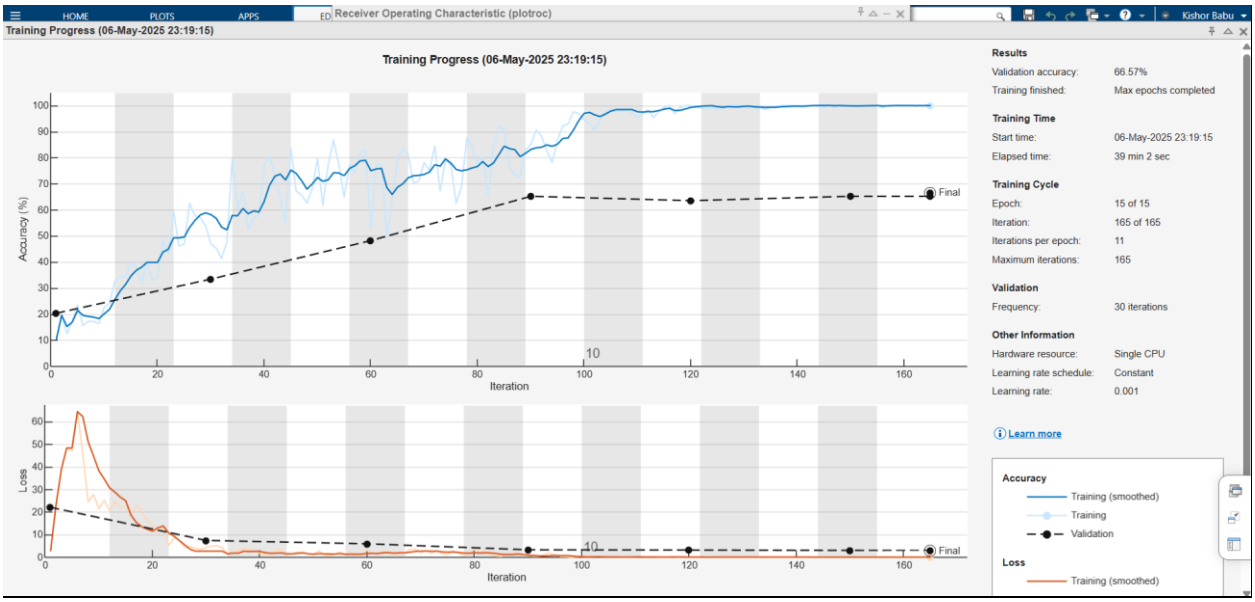
[redacted]

[redacted]

```
layers = [
    imageInputLayer(inputSize)

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,16,'Padding','same')
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,32,'Padding','same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(numel(unique(imds.Labels)))
    softmaxLayer
    classificationLayer];
```

[redacted]

[redacted]



[redacted]

[redacted]

```
options = trainingOptions('adam', ...
    'MaxEpochs', 15, ...
    'ValidationData', imdsVal, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
%% Train Network
figure; % To capture training-progress
net = trainNetwork(imdsTrain, layers, options);
```

[redacted]

[redacted]

[redacted]

[redacted]

Results	
Validation accuracy:	66.57%

[redacted]

[redacted]

[redacted]

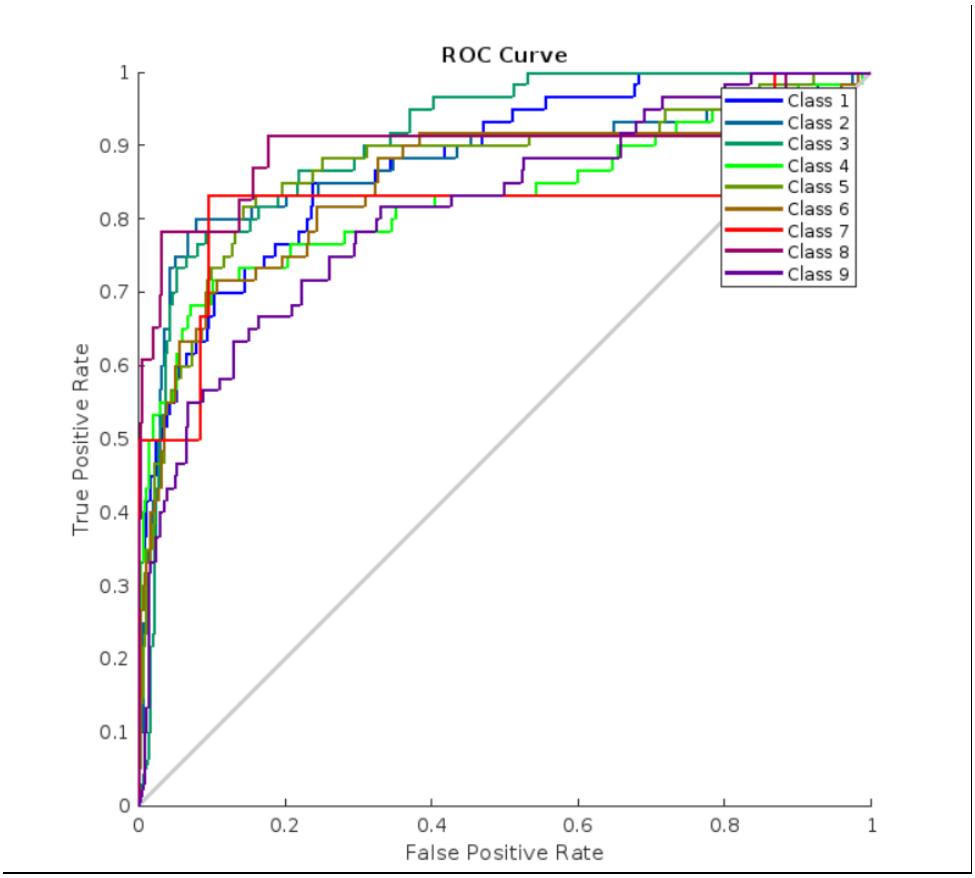
Confusion Matrix

True Class \ Predicted Class	Apta	Indian Rubber Tree	Karanj	Kashid	Nilgiri	Pimpal	Sita Ashok	Vad	Vilayati Chinch
Apta	31	7	7	2	7	3			3
Indian Rubber Tree	1	46	3		7				3
Karanj	2		46	3	3	4			2
Kashid	2	2	8	31	7	8			2
Nilgiri	3	3	3		38	5			8
Pimpal	3	6	2	1	1	39			8
Sita Ashok	1						3		2
Vad		2	2			6		9	4
Vilayati Chinch	1	4	6	1	5	10			33

[redacted]

[redacted]

[redacted]

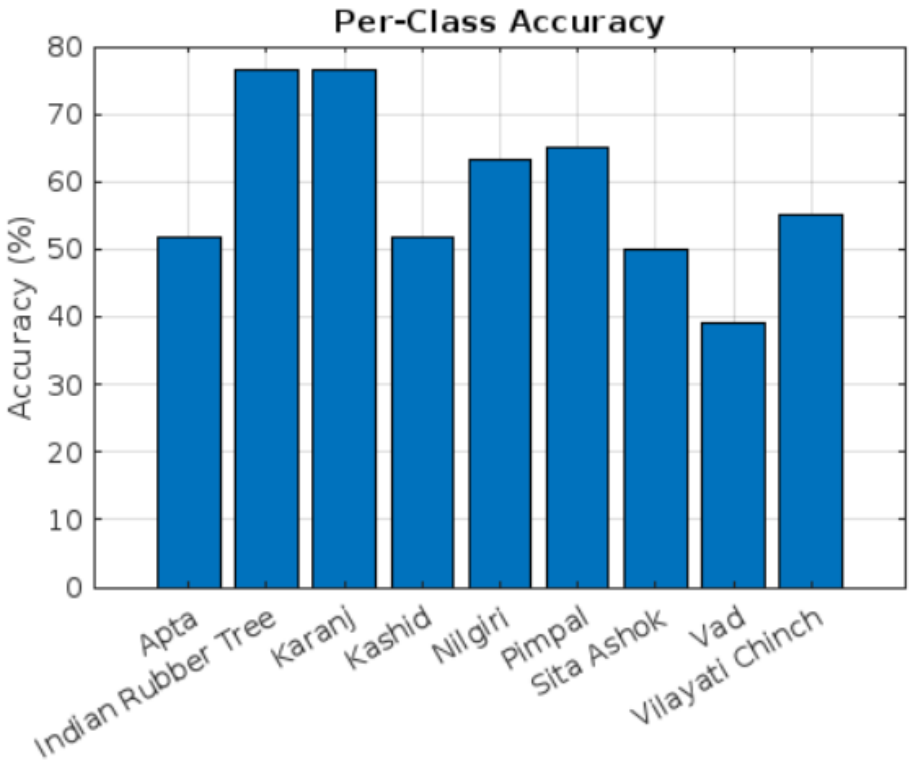


[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]